SuperMarket Express Reviewer

Nov 2, 2026 · @vowels = "aeiou" def contains_vowel(word): for letter in vow

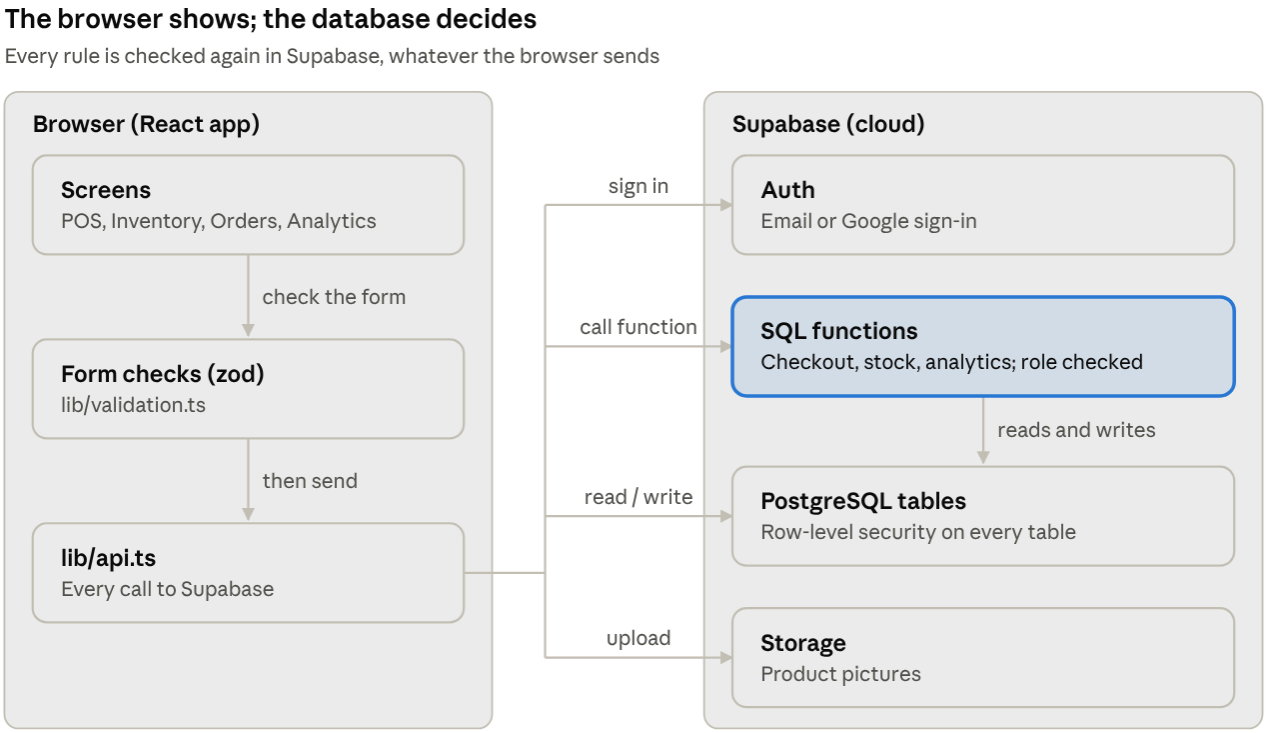
At a glance

SuperMarket Express is a web point-of-sale system for a small grocery: it sells, tracks stock by batch and expiry date, runs a loyalty program, and reports profit and loss. Everyone who signs up is a loyalty customer; an admin can also make someone a cashier or an admin.

What	Answer
Users	Customers (self-checkout), cashiers (till for any member), admins (everything)
Front end	React 18 + TypeScript, built with Vite, styled with Tailwind CSS v4 and Base UI components
Back end	Supabase: sign-in (email or Google), a PostgreSQL database, file storage for product pictures
Where the rules live	In the database: row-level security plus SQL functions for checkout, stock and analytics
Charts and PDFs	Recharts for charts; html-to-image + jsPDF for the receipt PDF
Form checks	zod schemas in `src/lib/validation.ts`

The main screens: the POS (product menu, basket, payment, receipt), Inventory (products, batches, receive stock, write-offs), Orders, Loyalty & Rewards, Users & Roles, and Sales Analytics.

Architecture



architecture · browser app and Supabase

The screens check forms first, then every request goes through `lib/api.ts` to Supabase, where the SQL functions and row-level security decide what is allowed.

Roles and access

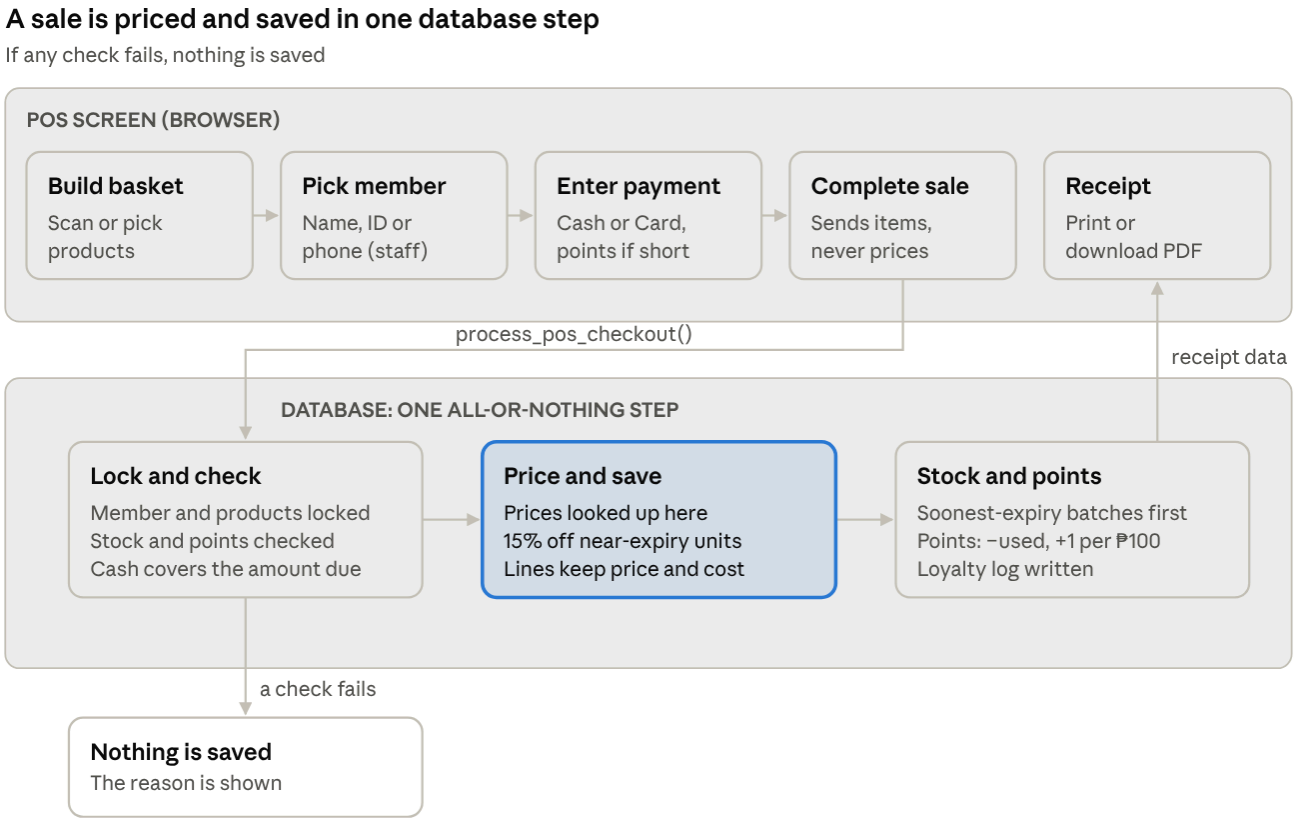
Every account is a customer; a row in the `cashier` or `admin` table adds that role. The app hides screens a role can't use, but the database is what actually enforces it.

Role	What they can do	Screen	Enforced in the database by
Customer	Buy for themselves (self-checkout), use their own points, see prices but never cost	`/self-serve`	Row-level security shows only their own loyalty row; the checkout function refuses a sale for anyone else
Cashier	Ring up a sale for any member, look up members, see stock (read-only)	`/cashier`, plus Customer	`is_staff()` checks in policies and functions

Role	What they can do	Screen	Enforced in the database by
Admin	Everything: products, receive and write off stock, orders, loyalty log, analytics, assign roles	`/admin`, plus Cashier and Customer	`is_admin()` checks; only admins can read `products.cost`

After signing in, a customer-only account goes straight to the shop. A cashier picks Customer or Cashier, and an admin picks any of the three. A person who types an address they can't use (for example `/admin`) is sent back to the role picker by `ProtectedRoute`.

Checkout flow



checkout flow · 5 screen steps, 3 database steps

The POS only previews prices and points; the database function `process_pos_checkout` re-checks, re-prices and saves the whole sale at once, then sends back the receipt.

Business rules

Each rule lives in one place in the database, and the screen only previews it. The browser never sends a price, so the database always re-prices the sale itself.

Rule	How it works	Where it lives
Earning points	1 point for every full ₱100 actually paid (₱199 earns 1, ₱200 earns 2), not counting the part paid with points	`process_pos_checkout_base`
Using points	Each point takes ₱5 off. Points can cover part of the bill (the rest by Cash or Card) or all of it (saved as paid with Loyalty Points)	`lib/loyalty.ts`, `POS.tsx`, checkout function
Near-expiry discount	Units from a batch expiring within 7 days are sold at 15% off	`lib/pricing.ts`, checkout function
Selling order	Units come from the batch with the soonest expiry first, unless a batch is picked	`order_items_allocate_batches` trigger
Expired stock	A batch expiring today or earlier counts as expired. A nightly job (12:05 AM Manila) removes it and logs the loss at cost	`process_expired_batches`, `spoilage_logs`
Expiry date	Required for every batch received, and must be after today	`receive_stock`, `lib/validation.ts`
Reorder level	When units in stock drop to this number or fewer, the product shows Low stock	`Inventory.tsx`
Frozen sale lines	Each order line keeps the price, cost and name it had at the moment of sale, so later edits don't change past orders or profit	`order_items_snapshot_product` trigger
Cash	Must cover the amount due; change = cash received − amount due	`cashReceivedSchema`, `process_pos_checkout`
Card	Charged exactly the amount due	`cardAmountSchema`

Rule	How it works	Where it lives
Every sale has a member	A sale must go on someone's loyalty account	checkout function
Member lookup	At the till a member is found by phone number or loyalty ID (name and email work too). Phones are saved in Loyalty & Rewards as PH mobile numbers (09XXXXXXXXX), one member per number	set_member_phone, POS.tsx
Unique SKU	No two products share a SKU. The Add and Edit forms warn while typing; a new product with the same name and brand as an existing one asks before saving	Inventory.tsx, unique rule on products.sku
Brand	Every product has a brand, shown on the POS menu, in the basket and in the product details, and searchable	products.brand

Database

Ten tables in Supabase's PostgreSQL database, plus Supabase's own `auth.users` for sign-in. The full diagram, every column and every relationship are in the SuperMarket Express Database ERD.

Table	Holds
`users`	Each account's email, name, phone and highest role
`customer`	One loyalty account per user: loyalty ID and points balance
`cashier`, `admin`	One row per person who holds that role
`products`	The catalogue: SKU, barcode, brand, name, category, price, cost (admins only), stock, reorder level, picture
`product_batches`	Each delivery of a product: quantity received and remaining, expiry date, status

Table	Holds
`orders`	One row per sale: member, totals, points used, payment method
`order_items`	The lines of a sale, with the price, cost and name frozen at the moment of sale
`loyalty_logs`	Every point earned or redeemed (the audit trail)
`spoilage_logs`	Stock written off as expired, damaged or spoiled, valued at cost

The main database functions the app calls (each checks the caller's role first):

Function	What it does
`process_pos_checkout`	Saves a sale in one all-or-nothing step and returns the receipt
`receive_stock`	Adds a batch with its expiry date and raises the product's stock
`write_off_stock`	Removes damaged or spoiled units and logs the loss
`process_expired_batches`	Takes expired stock off the shelves (also runs nightly)
`get_product_inventory`	Stock, near-expiry count and earliest expiry per product
`get_admin_sales_analytics`	The headline sales numbers
`get_profit_loss_series`	Orders, gross sales, costs and net income for a day, month or year
`get_sales_breakdown`	Payment mix and top products for any date range
`set_user_roles`	Gives or takes away the cashier and admin roles
set_member_phone	Saves a member's phone number (staff for anyone, others for themselves)

Triggers do work automatically when rows change: `products_assign_barcode` gives each new product a barcode, `order_items_allocate_batches` takes sold units off batches (soonest expiry first), and `order_items_snapshot_product` freezes each sale line's price, cost and name.

Security and validation

The browser is never trusted: every rule that matters is checked again inside the database, so a changed or fake browser can't get around it.

Layer	What it does	Example
Form checks (zod)	Clear messages before anything is sent: required fields, peso amounts with at most 2 decimals, SKU and category patterns, expiry after today, picture type and size	`newProductSchema`, `receiveStockSchema`, `cashReceivedSchema` in `lib/validation.ts`
Hidden screens	Each page is wrapped in `ProtectedRoute`, which sends people without the role back to the picker	A customer typing `/admin`
Row-level security	The database itself decides which rows each person may read or change	A customer reads only their own loyalty row; only admins read orders
Column privileges	Even a readable table can hide a column	`products.cost` is admin-only; everyone else gets prices
Database functions	Checkout, stock and analytics run as SQL functions that check the role first (`is_admin()`, `is_staff()`) and run with a fixed `search_path`	`process_pos_checkout` re-prices the whole sale itself
Transactions and locks	A sale is saved all-or-nothing; the member and products are locked while it runs	Two tills can't sell the same last unit

Sign-in uses Supabase Auth (email and password, or Google). The browser only holds the public "anon" key; what each person may do comes from their signed-in session and the rules above.

Practice questions

Question	Short answer
Why Supabase instead of writing your own server?	It gives sign-in, a PostgreSQL database and file storage out of the box. The business logic lives in SQL functions and row-level security, so there's no separate server to secure.
Can a customer change a price in the browser?	No. The browser only sends what is bought (product, quantity, batch). The checkout function looks up every price itself.
What if two cashiers sell the last unit at the same time?	Checkout locks the products (and the member) until the sale is saved. The second sale waits, then fails with "Not enough stock".
How does the 15% near-expiry discount work?	Units from a batch that expires within 7 days are charged 15% less. The POS shows it as a preview; the database applies it.
How do loyalty points work?	Earn 1 point per full ₱100 actually paid. Each point takes ₱5 off, for part of the bill (with Cash or Card) or all of it.
If a product's price or cost changes, does past profit change?	No. Each order line keeps its price, cost and name from the moment of sale (a trigger copies them).
What is the reorder level?	The low-stock alert: when units in stock drop to that number or fewer, the product shows Low stock so it gets reordered.
How is net income worked out?	Gross sales (money taken in) minus total cost, where total cost = cost of the goods sold + stock losses (expired, spoiled, damaged, valued at cost).
What happens to expired stock?	A batch expiring today or earlier is expired. A nightly job (or the button in Inventory) removes it and logs the loss at cost in `spoilage_logs`.
Why track batches instead of one stock number?	Each delivery has its own expiry date. Batches let the store sell the soonest-expiring units first, discount near-expiry ones, and value losses exactly.

Question	Short answer
Can a cashier open the admin screens?	No. The page sends them back, and even a direct request is refused by the database because they aren't an admin.
Why can't cashiers see cost?	Cost is business-private. The `cost` column is readable only by admins (column privileges), so it never reaches other browsers.
What if the sale saves but the receipt doesn't load?	The POS starts a fresh basket so the sale can't be charged twice, and says the order is in the Orders tab.
Which time zone is used?	Philippine time everywhere (`Asia/Manila`), for "today", expiry and analytics, in both the browser and the database.
How is the code checked?	TypeScript type-checking, zod form rules, and a production build; database changes are tried in a transaction that is rolled back before being applied.
Why have both a SKU and a barcode?	The spec asks for "SKUs/barcodes". The barcode (EAN-13, made by the database) is what the scanner reads; the SKU is the store's own short code that staff can read, type and search (e.g. DY-FRESHMILK-1L). Both are unique.
How does the cashier find a member by phone?	Typing the number in the member box finds them, with or without spaces or +63. Phones are added in Loyalty & Rewards and checked by the database: a PH mobile number, never shared by two members.

Glossary

Term	Meaning
Batch	One delivery of a product, with its own quantity and expiry date
FEFO	First expired, first out: sell the units that expire soonest first

Term	Meaning
Row-level security (RLS)	Database rules deciding which rows each signed-in person may see or change
RPC	"Remote procedure call": the app calling a database function by name, e.g. `supabase.rpc('receive_stock', …)`
SECURITY DEFINER	A database function that runs with its owner's rights, so it can do checked work the caller couldn't do directly
Trigger	Database code that runs by itself when a row is added or changed
Migration	A numbered SQL file that changes the database; applied in order with `npm run db:push`
Transaction	A group of changes saved all together or not at all
zod	The library that checks form values and produces the error messages
Props / state	In React: the inputs a component receives / the values it remembers between redraws
Hook	A React function starting with `use`, e.g. `useState`, `useEffect`, `useStoreData`
Portal	Drawing a pop-up straight into the page body, outside the screen it belongs to (the receipt does this)
Vite	The tool that runs the app while developing and builds it for the web
Tailwind CSS	Styling with small class names (`rounded-full`, `text-sm`) instead of separate CSS files
Gross sales	Money taken in from customers, after discounts and points
Net income	Gross sales minus cost of goods sold minus stock losses